

System Design — The Complete Study Guide

A single, self-contained study file distilled from [The System Design Primer](#) (donnemartin), its suggested video lectures, and additional research to fill the gaps the primer leaves open.

How to use this file: Read it top-to-bottom once for breadth. Then use the [Cheat Sheet](#) and [Flashcard Prompts](#) for review. Every section links back to the deeper source material in this repo ([README.md](#) , [solutions/...](#)) so you can drill down when a topic interests you.

The one idea to internalize: *Everything is a trade-off.* There is no "correct" architecture — only choices with costs. Interviewers are testing whether you can reason about those costs out loud.

Table of Contents

Part I — The Interview 1. [Mindset & what's actually being tested](#) 2. [The interview framework \(a repeatable 4-step process\)](#) 3. [Back-of-the-envelope estimation \(with worked examples\)](#)

Part II — Fundamentals & Trade-offs 4. [Performance vs. scalability](#) 5. [Latency vs. throughput](#) 6. [CAP & PACELC \(availability vs. consistency\)](#) 7. [Consistency patterns](#) 8. [Availability patterns \(+ the availability math\)](#)

Part III — The Building Blocks (your "menu" of components) 9. [DNS](#) 10. [CDN](#) 11. [Load balancers & reverse proxies](#) 12. [Application layer, microservices, service discovery](#) 13. [Databases — SQL scaling & NoSQL types](#) 14. [Caching](#) 15. [Asynchronism — queues & back pressure](#) 16. [Communication — HTTP, TCP/UDP, REST, RPC, gRPC, GraphQL, real-time](#)

Part IV — Deeper Distributed-Systems Topics (*gaps the primer marks "under development"*) 17. [Consistent hashing, indexing, rate limiting, quorum, idempotency, bloom filters, resilience, observability](#)

Part V — Practice & Reference - [Numbers every engineer should know](#) - [Worked practice problems \(the repo's 8 solutions, distilled\)](#) - [Object-oriented design questions](#) - [One-page cheat sheet](#) - [Self-test flashcard prompts](#) - [Study plan by timeline](#) - [Video lecture notes](#) - [Further reading map](#)

Part I — The Interview

1. Mindset & what is actually being tested

A system design interview is an **open-ended conversation that you are expected to lead**. Unlike a coding interview, there is no single right answer. The interviewer is evaluating:

- **Do you gather requirements before designing?** (Jumping to a solution is the #1 failure mode.)
- **Can you reason about trade-offs?** Every component you add has a cost — complexity, money, consistency, latency. Naming the cost is worth more than naming the component.
- **Can you estimate?** Rough numbers drive design decisions (do we need one box or a thousand? SQL or NoSQL?).
- **Do you know the building blocks** — load balancers, caches, queues, replicas, shards — and *when* each applies?
- **Can you identify bottlenecks and scale?** Start simple, then evolve the design under load.
- **Communication.** Think out loud, draw boxes, justify choices, invite feedback.

What you do NOT need: to know every technology, or to memorize a "correct" architecture. Breadth first, then depth in a few areas. More senior roles are expected to go deeper. (See the [Study guide table in the primer.](#))

Rule of thumb: *Start broad, go deep where prompted, and always say the trade-off out loud.*

2. The interview framework

Use this repeatable process for any "Design X" question. (Primer source: [How to approach a system design interview question](#).)

Step 1 — Scope: outline use cases, constraints, assumptions

Gather requirements. **Do not skip this.** Ask: - **Who** uses it and **how**? What are the core **use cases**? (Pick 2–3 to focus on; explicitly defer the rest.) - **How many users?** (DAU/MAU) **How many requests/sec?** - What is the **read:write ratio**? (Often 100:1 — this decides whether you optimize reads.) - What are the **inputs and outputs**? - **How much data?** Now and growing at what rate? - Latency / availability / consistency requirements? (Is stale data OK? Is downtime OK?)

Write the agreed scope down. This is your contract for the rest of the interview.

Step 2 — High-level design

Sketch only the components needed to make the use cases *work* end-to-end — typically `client → web/app tier → database` — plus anything **core to the design itself** (e.g. a queue + workers when async processing is the whole point, as in Mint's transaction extraction). Draw boxes and show how requests flow between them; justify each box.

Leave scaling infrastructure for Step 4. The test for what goes here vs. Step 4 is *functionality* vs. *scale*: is a component present so the system **works**, or so it **holds up under load**? Load balancer, CDN, cache, and read replicas are almost always there to *scale* → defer them to Step 4, where you add each under a profiled bottleneck. (The full `client → DNS → CDN → load balancer → ... → cache → database → queue` chain in [Part III](#) is a *menu* of the request path, not what you draw on the first pass.)

Step 3 — Design the core components

Go deep on the pieces that matter for *this* problem. Typically: - **API design** — define the key endpoints (method, path, params, response). This nails down the contract. - **Data model / schema** — tables (SQL) or key structures (NoSQL). Choose SQL vs NoSQL and say *why*. - **The core algorithm/trick** — e.g., for a URL shortener: hash generation + collision handling; for Twitter: timeline fan-out.

Step 4 — Scale the design

Identify bottlenecks under the load from Step 1, and address them with the [building blocks](#). Common moves: - **Load balancer** + **horizontal scaling** (stateless web tier). - **Caching** (the single biggest lever for read-heavy systems). - **CDN** for static/media content. - **Database scaling** — read replicas → federation → sharding → denormalization. - **Asynchronism** — move slow work to queues/workers. Always name the trade-off (complexity, consistency, cost) for each move.

A reusable template: *Requirements → estimates → API → data model → high-level diagram → deep-dive → identify bottlenecks → scale each tier → summarize trade-offs.*

3. Back-of-the-envelope estimation

Interviewers often ask you to size the system by hand. You only need a few reference numbers (see [Numbers every engineer should know](#)) and a method.

The method: 1. Start from **users** (DAU) and **actions per user per day**. 2. Convert to **QPS**: $\text{QPS} \approx \text{daily_count} / 86,400$. A day is ~**100,000 seconds** (86,400, round up). Handy shortcut: **1 million/day $\approx$ 12 QPS**. 3. **Peak QPS** $\approx$ 2–3 $\times$ average (spikes). Some use up to 10 $\times$ for bursty workloads. 4. **Storage** = objects/day $\times$ bytes/object $\times$ retention ($\times$ replication factor). 5. **Bandwidth** = QPS $\times$ bytes/response. 6. Round aggressively with **powers of two**. Precision is not the point — the *order of magnitude* drives the design.

What to compute for THIS problem (don't compute everything). Estimation always lives at the *end of Step 1 (Scope)* — never its own step: **use cases** $\rightarrow$ **state assumptions** $\rightarrow$ **calculate usage**. The *method* above is identical every time; what changes is *which output you spotlight*, chosen by working backward from "what decision am I about to make?" Compute a number only if it changes a design choice, then **stop at one order of magnitude** — check it against the **capacity numbers** (e.g. 2,000 writes/s vs. a single SQL master's ~1,000) to trigger the next move.

Problem	The number that matters	What it decides
Mint	2,000 writes /s, write-heavy	one SQL master can't cope $\rightarrow$ shard / queue
Twitter	600k timeline inserts/s (fan-out)	the celebrity / hybrid problem
URL shortener	6 billion keys	base62 key length (6 vs 7 chars)
Sales rank	hourly batch, 40k reads/s	MapReduce offline, tiny serve table

Mint skips bandwidth/CDN math (not media-heavy $\rightarrow$ wouldn't change anything); Twitter leans on it. That selectivity is the skill — it's not inconsistency between the worked examples.

(Two fully worked examples — a Twitter-scale service and a URL shortener — are in [§17 Deeper Topics / estimation](#) once integrated below, and in the [practice problems](#). Learn the method, not the numbers.)

Part II — Fundamentals & Trade-offs

These are the "physics" of system design. Master these six trade-offs and most design decisions become explainable.

4. Performance vs. scalability

- A service is **scalable** if adding resources yields a *proportional* increase in performance (more work served, or larger datasets handled).
- **The diagnostic:**
- **Performance problem** $\rightarrow$ your system is slow *for a single user*.
- **Scalability problem** $\rightarrow$ fast for one user, but slow *under load*.

Source: [primer §Performance vs scalability](#), [A word on scalability](#) (Werner Vogels).

5. Latency vs. throughput

- **Latency** = time to perform one action (e.g., ms per request).
- **Throughput** = number of actions per unit time (e.g., requests/sec).
- **Goal:** maximize **throughput** while keeping **latency acceptable**.
- They trade off: batching improves throughput but can raise latency; the two are linked by **Little's Law** ($\text{concurrency} = \text{throughput} \times \text{latency}$).

Source: [primer §Latency vs throughput](#).

6. CAP & PACELC

CAP theorem

In a distributed system you can guarantee only **two of three**: - **Consistency (C)** — every read sees the most recent write (or an error). - **Availability (A)** — every request gets a (non-error) response, but maybe stale. - **Partition tolerance (P)** — the system keeps working despite dropped/delayed messages between nodes.

The crucial nuance interviewers want: networks *will* partition, so **P is not optional**. The real decision is: **when a partition happens, do you choose C or A?** - **CP** — on partition, refuse/err to avoid serving stale data. Choose when you need atomic reads/writes (banking, inventory). Examples: traditional RDBMS with sync replication, HBase, Zookeeper, etcd. - **AP** — on partition, keep serving from whatever node answers (possibly stale); reconcile later. Choose when availability + [eventual consistency](#) are acceptable (social feeds, shopping carts). Examples: Cassandra, DynamoDB, CouchDB.

PACELC — the modern refinement

CAP only says what happens *during* a partition. **PACELC** adds the normal case:

If Partition (P): choose A or C. Else (E): choose Latency (L) or Consistency (C).

Even with no partition, synchronous strong consistency costs latency (coordination round-trips). So the everyday trade-off is often **latency vs. consistency**. E.g., DynamoDB/Cassandra are **PA/EL** (favor availability and latency); fully consistent systems like HBase/BigTable are **PC/EC**.

Sources: [primer §CAP](#), [CAP theorem revisited \(Greiner\)](#). (Video notes: [CAP lecture](#).)

7. Consistency patterns

With multiple copies of data, you choose how synchronized clients' views are:

- **Weak consistency** — after a write, reads *may or may not* see it. Best-effort. Good for real-time systems where stale data is useless anyway: VoIP, video chat, multiplayer games, memcached.
- **Eventual consistency** — after a write, reads will see it *eventually* (usually ms). Data is replicated **asynchronously**. Powers highly available systems: DNS, email, most NoSQL, social feeds.
- **Strong consistency** — after a write, reads see it immediately. Data is replicated **synchronously**. Needed for transactions: RDBMSes, file systems.

Trade-off: stronger consistency ⇒ more coordination ⇒ higher latency and lower availability. Source: [primer §Consistency patterns](#).

8. Availability patterns

Two complementary tools: **fail-over** and **replication**.

Fail-over

- **Active-passive (master-slave):** heartbeats between an active server and a passive standby. If the heartbeat stops, the passive takes over the active's IP. Downtime depends on whether standby is "hot" (running) or "cold" (needs boot).
- **Active-active (master-master):** both servers handle traffic and share load. Needs both IPs known (via DNS or app logic).
- **Cost:** more hardware, more complexity, and risk of **data loss** if the active dies before replicating its latest writes.

Availability in numbers ("nines")

Availability is measured in **9s**. Memorize the shape:

Availability	Downtime/year	Downtime/day
99.9% (three 9s)	~8h 46m	~1m 26s
99.99% (four 9s)	~52m 36s	~8.6s
99.999% (five 9s)	~5m 15s	~0.86s

Composing availability: - **In sequence** (request must pass through both): $A_{total} = A_{foo} \times A_{bar}$. Two 99.9% components in series → **99.8%** (worse — chains hurt). - **In parallel** (redundant, either can serve): $A_{total} = 1 - (1 - A_{foo}) \times (1 - A_{bar})$. Two 99.9% components in parallel → **99.9999%** (much better — redundancy helps).

Takeaway: every hop in series *lowers* availability; redundancy in parallel *raises* it. This is why you eliminate single points of failure. Source: [primer §Availability patterns](#).

Part III — The Building Blocks

Think of these as a **menu**. In an interview you assemble a system by picking components and justifying each. For every block below, know: *what it does, when to use it, and its main disadvantage*.

Typical request path:

```
Client → DNS → CDN (static) → Load Balancer → Web Server (reverse proxy)
      → Application / Microservices → Cache ≠ Database
      ↘ Message Queue → Workers
```

9. DNS

Translates a domain (`www.example.com`) to an IP. **Hierarchical** with authoritative servers at the top; results are **cached** (browser/OS/resolver) per a **TTL**.

Record types: `A` (name → IP), `CNAME` (name → another name), `NS` (name servers), `MX` (mail).

Managed DNS (Route 53, Cloudflare) can also **route traffic**: weighted round-robin (A/B testing, drain a server for maintenance), latency-based, geolocation-based.

Disadvantages: small lookup delay (mitigated by caching); DNS can be a DDoS target (the 2016 Dyn attack took down Twitter/GitHub for many).

Source: [primer §DNS](#).

10. CDN

A globally distributed network of proxy servers that serve content from a location **near the user**. Great for static assets (HTML/CSS/JS, images, video); some (CloudFront) also serve dynamic content.

Two flavors: - **Push CDN** — *you* upload content to the CDN when it changes. You control expiry. Good for **low-traffic** or rarely-changing sites (content placed once, not re-pulled). Maximizes storage, minimizes redundant traffic. - **Pull CDN** — CDN fetches content from your origin on the **first user request**, then caches it for a TTL. First request is slower; good for **high-traffic** sites (only popular content is cached, traffic spreads out).

Disadvantages: cost; content can be **stale** until TTL expires; you must rewrite URLs to point at the CDN.

Source: [primer §CDN](#).

11. Load balancers & reverse proxies

Load balancer

Distributes incoming requests across multiple servers. Benefits: - Prevents routing to **unhealthy** servers (health checks). - Prevents **overload** of any one resource. - Helps eliminate **single points of failure**. - **SSL termination** (offload decrypt/encrypt from backends). - **Session persistence** (sticky sessions via cookies) if the app isn't stateless.

Routing algorithms: random, least-loaded, round-robin / weighted round-robin, session/cookie-based, or by **Layer 4 / Layer 7**.

- **Layer 4** LB — routes on **transport-level** info (source/dest IP + port), doesn't inspect content. Faster, less flexible. Uses NAT.
- **Layer 7** LB — routes on **application-level** info (URL path, headers, cookies). Can send `/video/*` to video servers and `/billing/*` to hardened servers. More flexible, slightly more work.

To avoid the LB itself being a single point of failure, run **multiple LBs** (active-passive or active-active).

Horizontal vs. vertical scaling (the LB enables horizontal): - **Vertical (scale up)**: bigger box. Simple, but has a ceiling, gets expensive fast, and is still a single point of failure. - **Horizontal (scale out)**: more commodity boxes behind an LB. Cheaper, more available, but adds complexity and **requires stateless servers** — no local session/user data. Store sessions in a shared store (Redis, DB).

Reverse proxy

A web server (NGINX, HAProxy) that fronts your backends and presents one unified interface to clients. Benefits: **security** (hide backends, block IPs, rate-limit), **SSL termination**, **compression**, **caching**, **serving static content**.

LB vs. reverse proxy: an LB spreads traffic across *many* servers doing the same job; a reverse proxy is useful even with *one* backend (for the security/caching/SSL benefits). NGINX/HAProxy do both.

Disadvantages (both): added complexity; a single LB/proxy is itself a SPOF (so you run more than one, adding more complexity).

Source: [primer §Load balancer](#), [§Reverse proxy](#).

12. Application layer, microservices, service discovery

Separate the web tier from the application (platform) tier so each scales independently. Add an API → add app servers without adding web servers. Follows the **single-responsibility principle**: small, autonomous services.

- **Microservices** — a suite of independently deployable, small, modular services, each running one process and communicating over a lightweight mechanism. E.g., Pinterest: user-profile, follower, feed, search, photo-upload services. Trade-off: operational and deployment complexity (many moving parts, distributed debugging, network calls).
- **Service discovery** — services find each other via a registry of names/addresses/ports: **Consul**, **etcd**, **Zookeeper**. Includes **health checks** (often an HTTP endpoint) and a built-in key-value store for shared config.

Source: [primer §Application layer](#).

13. Databases

The database is usually the **hardest thing to scale** (state is heavy). Know the RDBMS scaling techniques and the NoSQL family.

RDBMS (SQL) — ACID

Data in tables; transactions guarantee **ACID**: - **Atomicity** — all-or-nothing. - **Consistency** — a transaction moves the DB from one valid state to another. - **Isolation** — concurrent transactions behave as if serialized. - **Durability** — once committed, it stays committed.

Five ways to scale a relational DB:

1. **Master-slave replication** — master takes reads+writes and replicates to read-only slaves. Scales **reads**. On master failure you promote a slave (needs extra logic).
2. **Master-master replication** — multiple masters take reads+writes. Improves write availability, but you need to route writes (LB/app logic), and you get **conflict resolution** problems and either loose consistency or higher write latency. - *Replication downsides (both)*: potential data loss if master dies pre-replication; **replication lag** grows with more slaves; replays can bog down slaves; more hardware/complexity.
3. **Federation (functional partitioning)** — split DBs **by function** (e.g., `users`, `products`, `forums`). Less traffic per DB, less replication lag, more cache locality, parallel writes. Downside: cross-function **joins** get hard; app must route to the right DB; ineffective if one function is itself huge.
4. **Sharding** — split a single logical table's **rows** across DBs (e.g., by user last-name initial or geography). Less traffic/replication per shard, smaller indexes → faster queries, one shard down ≠ all down. Downsides: app complexity; **hot spots** (uneven load — mitigate with [consistent hashing](#)); cross-shard **joins** hard.
5. **Denormalization** — write redundant copies to avoid expensive joins; trade write cost + duplication for faster reads. Great when reads outnumber writes 100:1. Materialized views (PostgreSQL/Oracle) automate it. Downside: data duplication, sync complexity, worse under heavy writes.

Plus **SQL tuning**: benchmark (`ab`) + profile (slow query log); tighten schema (`CHAR` vs `VARCHAR` , right int sizes, `NOT NULL`); add **indexes** (B-tree — logarithmic lookups, but slow writes and cost RAM); avoid expensive joins (denormalize); partition hot tables.

NoSQL — BASE

Data as **key-value**, **document**, **wide-column**, or **graph**. Denormalized; joins done in app code; most lack full ACID and favor **eventual consistency**.

BASE (vs ACID): **B**asically **A**vailable, **S**oft state, **E**ventual consistency — chooses availability over consistency.

Type	Abstraction	Use for	Examples
Key-value	hash table (O(1) r/w, memory/SSD)	caches, session data, simple/hot data	Redis, Memcached, DynamoDB
Document	key-value where value is a document (JSON/XML)	flexible/occasionally-changing schemas, query by document fields	MongoDB, CouchDB, DynamoDB, Elasticsearch
Wide-column	nested map <code>ColumnFamily<RowKey, Columns<ColKey, Value, Timestamp>></code>	very large datasets, high write throughput, high availability	Cassandra, HBase, Bigtable
Graph	graph (nodes + edges)	complex many-to-many relationships (social graphs, recommendations)	Neo4j, FlockDB

SQL or NoSQL? (a decision checklist)

Reach for SQL when: structured data, strict schema, relational data, complex joins, **transactions**, clear scaling patterns, fast index lookups, mature tooling.

Reach for NoSQL when: semi-structured / flexible schema, non-relational data, no complex joins, TB–PB scale, very high write/IOPS throughput. Well-suited data: clickstream/logs, leaderboards, shopping carts (temporary), hot lookup tables, metadata.

Interview soundbite: "Reads or writes dominate? Do we need transactions/joins or flexible scale? That decides SQL vs NoSQL." (Then justify — don't just say "NoSQL because scale.")

Source: [primer §Database](#). (Video notes: [Intro to NoSQL](#).)

14. Caching

Caching is the **highest-leverage optimization** for read-heavy systems. It cuts load on servers/DBs and speeds up responses. A cache in front of the DB absorbs uneven load and hot-key spikes.

Where you can cache (client → server): - **Client** (browser/OS), **CDN**, **web server** (reverse proxy / Varnish), **database** (built-in), **application** (Redis/Memcached — in-RAM, much faster than disk).

What to cache (two granularities): - **Query-level:** hash the query as key → store result. Simple, but **hard to invalidate** (one changed cell may invalidate many cached queries). - **Object-level:** cache assembled application objects (like class instances). Easier to reason about; remove on underlying change; enables async assembly. **Prefer this.** Good candidates: user sessions, rendered pages, activity streams, user-graph data.

Cache update strategies (know all four + the trade-off):

Strategy	How it works	Pros	Cons
Cache-aside (lazy loading)	App checks cache; on miss, reads DB, populates cache.	Only requested data cached; resilient to cache failure.	3 trips on miss; data can go stale (fix with TTL); cold cache after node replace.
Write-through	App writes to cache; cache synchronously writes DB.	Cache never stale; reads of just-written data fast.	Write latency higher; new nodes miss until data re-written; may cache unread data.
Write-behind (write-back)	App writes to cache; cache writes DB asynchronously .	Fast writes.	Data loss if cache dies before flush; more complex.
Refresh-ahead	Cache proactively refreshes popular entries before they expire.	Reduced latency if predictions are good.	Bad predictions waste work / hurt performance.

The hard part — cache invalidation ("one of the two hard problems in CS"). You must keep cache consistent with the source of truth; use **TTLs**, explicit invalidation, or write-through. LRU (least-recently-used) is the common eviction policy for full caches. Redis adds persistence and rich data structures (sorted sets, lists).

Source: [primer §Cache](#).

15. Asynchronism

Move slow or non-urgent work **out of the request path** so users aren't blocked. Also used for scheduled/aggregation work done in advance.

- **Message queues** (Redis, RabbitMQ, Amazon SQS, **Kafka**) — receive, hold, deliver messages. Flow: app **publishes** a job and immediately tells the user "received"; a **worker** later picks it up, processes, and signals completion. Example: posting a tweet returns instantly; fan-out to followers happens in the background.
- **Broker trade-offs:** Redis = simple but messages can be lost; RabbitMQ = popular AMQP, you manage nodes; SQS = hosted but higher latency and possible **duplicate delivery** (design for [idempotency](#)); Kafka = high-throughput durable log, great for event streaming.
- **Task queues** (Celery) — receive tasks + data, run them, return results; support scheduling and heavy background jobs.
- **Back pressure** — if the queue grows faster than it drains, cap its size. When full, reject new work fast (HTTP **503**), prompting clients to **retry with exponential backoff**. This preserves throughput and latency for jobs already queued

instead of collapsing.

Disadvantage: for cheap or truly real-time operations, queues add latency and complexity — keep them synchronous.

Source: [primer §Asynchronism](#).

16. Communication

OSI layers (just enough)

HTTP rides on **TCP/UDP** (transport), which ride on **IP** (network). Load balancing and routing happen at Layer 4 (transport) or Layer 7 (application).

HTTP

A request/response protocol. Request = **verb + resource**. Self-contained, so it flows through proxies/caches/LBs.

Verb	Meaning	Idempotent	Safe	Cacheable
GET	read	✓	✓	✓
POST	create / process	✗	✗	only w/ freshness info
PUT	create/replace	✓	✗	✗
PATCH	partial update	✗	✗	only w/ freshness info
DELETE	delete	✓	✗	✗

Idempotent = calling it many times has the same effect as once (matters for safe retries).

TCP vs. UDP

- **TCP** — connection-oriented (handshake), **reliable**: sequence numbers + checksums, acknowledgements + retransmission, **flow control** and **congestion control**. In-order, no corruption — but slower, higher overhead, and open connections consume memory (use **connection pooling**). Use for: web, DB, SMTP, FTP, SSH — anything needing every byte intact.
- **UDP** — connectionless, **unreliable**: datagrams may arrive out of order or not at all; no congestion control; can **broadcast**. Leaner and lower-latency. Use for: VoIP, video chat, streaming, games, DHCP — where **late data is worse than lost data**.

Pick TCP when you need all data intact. **Pick UDP** when you need lowest latency and can tolerate loss.

RPC vs. REST

- **RPC** (gRPC, Thrift, Avro, Protobuf) — call a procedure on a remote server as if local (client stub marshals args → server stub executes). **Exposes behaviors**. Great for **internal**, performance-sensitive service-to-service calls. Downsides: clients **tightly coupled** to the service; a new API per operation; harder to debug; caching is awkward.
- **REST** — a stateless, cacheable client/server style over resources (URIs) manipulated with HTTP verbs. **Exposes data**. Qualities: identify resources by URI, act via verbs, self-descriptive errors (status codes), HATEOAS. Great for **public APIs**; scales horizontally (stateless). Downsides: awkward for non-hierarchical actions; fixed verb set; **over-/under-fetching** (multiple round trips or bloated payloads for mobile).

Modern protocols (fill-the-gap — see §17 for detail)

- **gRPC** — RPC over HTTP/2 with Protobuf; binary, streaming, low-latency; the default for internal microservice calls.

- **GraphQL** — client specifies exactly the fields it needs in one query; fixes REST over/under-fetching; great for varied mobile/UI clients. Cost: server complexity, caching harder.
- **Real-time: WebSockets** (full-duplex, e.g. chat), **Server-Sent Events** (server – client stream, e.g. feeds/notifications), **long polling** (fallback).

Sources: [primer §Communication](#), [§REST/RPC comparison](#).

Security (basics)

Encrypt **in transit and at rest**; **sanitize all inputs** (prevent XSS/SQL injection); use **parameterized queries**; apply **least privilege**. See [primer §Security](#).

Part IV — Deeper Distributed-Systems Topics

17. Deeper Topics

These fill the gaps the primer marks "under development" or leaves thin. They're the topics that separate a "junior" answer from a "senior" one. Each ends with an **interview soundbite** you can say almost verbatim.

17.1 Consistent hashing

The problem. To spread K keys over N nodes, the naive scheme is $\text{node} = \text{hash}(\text{key}) \bmod N$. Add or remove one node and N changes, so almost every key remaps ($\approx (N-1)/N$ of them). For a cache that's a total cache-miss storm; for a sharded store it reshuffles nearly all data.

The hash ring. Map the hash output onto a circle (e.g. $0 \dots 2^{32}-1$, wrapping). Hash each **node** (by id/IP) to points on the ring and each **key** to a point. A key is owned by the first node found walking **clockwise**. Adding/removing a node only affects the arc between it and its neighbor — everything else keeps its owner, so only **$\sim K/N$ keys move** instead of $\sim K$.

Virtual nodes (vnodes). With few nodes the arcs are lumpy → uneven load, and a departing node dumps its whole load on one neighbor. Give each physical node many ring points (e.g. 100–200 vnodes). This smooths load (law of large numbers), spreads a failed node's keys across many neighbors, and lets you weight bigger machines (more vnodes).

Where it's used: Dynamo, Cassandra, Riak, memcached clients (Ketama), CDNs, Discord. Caveat: it balances *key count*, not *access frequency* — a hot key still overloads its owner (needs separate hot-key handling).

Soundbite: "Consistent hashing puts nodes and keys on a hash ring, so a node join/leave only remaps $\sim K/N$ keys instead of nearly all of them — and virtual nodes keep the load even."

17.2 Database indexing internals

An index trades write/space overhead for faster reads. Two dominant families:

B-tree / B+-tree (read-optimized — the RDBMS default). A balanced, high-fanout sorted tree; each node $\approx$ one disk page. Shallow depth (3–4 levels indexes billions of rows) → **$O(\log n)$ lookups**. In a **B+-tree**, all values live in linked leaf nodes → excellent **range scans** and ordered reads. Updates happen **in place**. Used by PostgreSQL, MySQL/InnoDB, Oracle. Best for read-heavy/balanced OLTP.

LSM-tree + SSTables (write-optimized). Writes append to a log and an in-memory **memtable**; when full it flushes as an immutable sorted **SSTable** file. Writes are sequential → very high write throughput. Reads check memtable then SSTables newest → oldest (slower — mitigated by **Bloom filters** per SSTable). Background **compaction** merges files and drops tombstones. Used by Cassandra, RocksDB, LevelDB, HBase. Best for write-heavy ingest (logs, time-series, events).

Clustered vs secondary index. A **clustered** index stores the rows physically in key order (the leaf is the row) — one per table, fastest key lookups (InnoDB clusters on the PK). A **secondary/non-clustered** index maps a column to a row locator, needing a second fetch. A **covering index** includes every column a query needs → index-only scan, no row fetch.

Soundbite: "B-trees update in place and are read/range-optimized (RDBMS default); LSM-trees turn writes into sequential SSTable appends for huge write throughput, paying it back in read amplification and compaction."

17.3 Rate limiting

Caps request rate to protect services, enforce quotas, and stop abuse.

Algorithm	How it works	Trade-off
Token bucket	Bucket holds $\leq B$ tokens, refills at r/sec ; each request spends one.	Allows bursts up to B , cheap (count + timestamp). Most popular.
Leaky bucket	Requests queue and drain at a fixed rate; overflow dropped.	Smooths output to constant rate, but adds queuing latency, no bursts.
Fixed window	Count per fixed window (per minute), reset at boundary.	Trivial, but a boundary spike can allow $2\times$ the limit across an edge.
Sliding window log	Store every request timestamp; count those in the last window.	Exact, but memory-heavy.
Sliding window counter	Weighted blend of current + previous window counts.	Approximates the log in $O(1)$ memory, kills the boundary spike.

Distributed (Redis): centralize the counter so a fleet enforces a global limit. Use an atomic **Lua script** (or `INCR + EXPIRE`) for token bucket / fixed window, or a **sorted set** (`ZADD / ZREMRANGEBYSCORE / ZCARD`) for sliding-window-log. Return **429 Too Many Requests** with `Retry-After`. Watch the Redis hot-key/SPOF risk (shard by user).

Soundbite: "Token bucket for burst-tolerant average rate, sliding-window counter to kill the fixed-window boundary spike — and push the counter into Redis with an atomic Lua script for distributed enforcement."

17.4 Modern communication protocols

REST vs gRPC vs GraphQL:

	REST	gRPC	GraphQL
Style	Resources + HTTP verbs, JSON	RPC over HTTP/2 + Protobuf (binary)	Query language, single endpoint
Streaming	No	Yes (uni + bidi)	Subscriptions (over WS)
Best for	Public APIs , CRUD, caching, ubiquity	Internal service-to-service , low latency, polyglot	Aggregating resources for diverse clients (mobile/web)
Weakness	Over/under-fetching, N round-trips	Weak in browsers, not human-readable	Caching harder, $N+1$ resolvers, query-cost risk

Real-time / server push:

	Long polling	SSE	WebSockets
Direction	Client re-requests	Server → client (one-way)	Full-duplex
Best for	Legacy fallback	Feeds, notifications, live scores, LLM token streaming	Chat, gaming, collaborative editing, trading

Rule: one-way push → **SSE** (simple, auto-reconnect, plain HTTP); two-way → **WebSockets**; long polling only as a fallback.

HTTP/2 multiplexes streams over one TCP connection with header compression (fixes app-layer head-of-line blocking).

HTTP/3 (over QUIC/UDP) removes TCP-level HOL blocking and adds faster 0/1-RTT setup + connection migration.

Soundbite: "REST for public/CRUD, gRPC for internal high-throughput service-to-service, GraphQL to let clients fetch exactly what they need — and for real-time, SSE for one-way push, WebSockets for full-duplex."

17.5 Idempotency & delivery semantics

Delivery guarantees over an unreliable network: - **At-most-once** — send, don't retry. Can *lose* messages. OK for metrics. - **At-least-once** — retry until acked. No loss, but **duplicates** (the common default: Kafka, SQS). - **Exactly-once** — impossible at the pure delivery layer (the network can always drop an ack).

"Effectively-once" = at-least-once delivery + **idempotent processing/dedup**, so reprocessing a duplicate is a harmless no-op.

Idempotency keys: client attaches a unique key (UUID); server records processed keys and returns the stored result on a retry instead of re-executing. Needs an atomic check-and-record (unique constraint / conditional write) + expiry window. Make operations naturally idempotent where possible (`SET balance = x` is idempotent; `balance += x` is not).

Interview relevance: payments (Stripe's `Idempotency-Key` prevents double charges), queue consumers (must be idempotent because at-least-once *will* redeliver), inventory decrements, "send email once."

Soundbite: "You can't get exactly-once delivery over a lossy network, so you get it in effect: at-least-once delivery plus idempotency keys so reprocessing a duplicate is a safe no-op."

17.6 Quorum, replication & consensus

Quorum: with N replicas, require W acks per write and R replicas per read. If $W + R > N$, read and write sets overlap → a read sees the latest write (strong consistency). Balanced default $N=3, W=2, R=2$ tolerates one node down. $W+R \leq N$ → eventual consistency, possible stale reads. (Dynamo/Cassandra tunable consistency.)

Leader-based replication: all writes to one leader, replicated to followers. **Synchronous** = no data loss on failover but higher latency; **asynchronous** = fast but may lose un-replicated writes. Followers scale reads (at the cost of replication-lag staleness). Alternatives: multi-leader (multi-region, conflict resolution), leaderless (Dynamo quorums).

Consensus (Paxos / Raft): let a cluster agree on an ordered log despite failures — for leader election, replicated logs, config/metadata. **Raft** is the understandable one (etcd, Consul, CockroachDB). Needs a majority quorum → tolerates $\lfloor (N-1)/2 \rfloor$ failures. Say: "I'd keep cluster membership/config in a Raft-backed store like etcd or ZooKeeper."

One-liners: **Read repair** (fix stale replicas on the read path) · **Hinted handoff** (a stand-in stores writes for a down node and forwards them on recovery) · **Vector clocks** (per-replica counters to detect concurrent/conflicting updates vs. one strictly newer).

Soundbite: " $W + R > N$ gives quorum overlap for strong consistency; when I need a fault-tolerant single source of truth I reach for a Raft/Paxos-backed system like etcd or ZooKeeper."

17.7 Bloom filters

A compact probabilistic set: a bit array + k hash functions. **Add:** set the k bits. **Query:** if any of the k bits is 0 → **definitely not present**; if all 1 → **probably present**. Key property: **no false negatives, possible false positives**. Can't delete from a basic filter (use a counting Bloom filter). A few bits per element, $O(k)$ time.

Canonical uses: (1) **LSM engines** (Cassandra/RocksDB/HBase) — skip SSTables that can't contain a key; (2) **caches/CDNs** — avoid pointless lookups for keys that don't exist ("one-hit-wonder" filtering); (3) **web crawler/dedup** — "have I seen this URL?" A false positive just rarely skips something, which is acceptable.

Soundbite: "A Bloom filter cheaply answers 'definitely not present or maybe present' with no false negatives — perfect for skipping expensive lookups like SSTable reads in Cassandra."

17.8 Back-of-the-envelope estimation — worked examples

Reference: 1 day $\approx 86,400$ s $\approx 10^5$ s. 1M/day ≈ 12 QPS. Round aggressively.

Example A — Twitter-like service. Assume 300M MAU, 50% daily active $\rightarrow$ **150M DAU**, 2 tweets/user/day, tweet = 300 B text (+ media on 20% at ~ 1 MB), read:write = 100:1.

- **Writes:** $150\text{M} \times 2 = 300\text{M}$ tweets/day $\div 10^5$ s $\approx$ **3,000 writes/s** (avg); peak $\sim 2\times \rightarrow$ **6,000/s**.
- **Reads:** $100 \times 3,000 =$ **300,000 reads/s** (avg); peak $\rightarrow$ **600,000/s**.
- **Fan-out (push model):** avg ~ 200 followers $\rightarrow 3,000 \times 200 \approx$ **600,000 timeline inserts/s**. This is why celebrities break pure fan-out-on-write $\rightarrow$ hybrid (pull for high-follower accounts).
- **Storage (text):** $300\text{M} \times 300$ B $\approx$ **90 GB/day** $\rightarrow$ **~ 33 TB/year**.
- **Storage (media):** $20\% \times 300\text{M} \times 1$ MB = 60 TB/day $\rightarrow$ **~ 22 PB/year** (media dominates).
- **Bandwidth:** read egress for media $\approx$ **tens of GB/s** at the edge $\rightarrow$ must serve from a **CDN**.

Example B — URL shortener. Assume 100M new URLs/month, read:write = 100:1, 5-year horizon.

- **Writes:** month $\approx 2.5\text{M}$ s $\rightarrow 100\text{M} / 2.5\text{M} \approx$ **40 writes/s** (avg); peak $\rightarrow$ **80/s**.
- **Reads:** $100 \times 40 =$ **4,000 reads/s** (avg); peak $\rightarrow$ **8,000/s**.
- **Storage (5 yr):** $100\text{M} \times 12 \times 5 =$ **6 billion URLs** $\times \sim 500$ B $\approx$ **~ 3 TB** (cacheable in memory).
- **Base62 key length:** 62 chars, need $\geq 6 \times 10^9$ keys. $62^6 \approx 57$ billion > 6 billion $\rightarrow$ **6 chars suffice**; use **7** ($62^7 \approx 3.5$ trillion) for headroom.

Method recap: (1) period $\rightarrow$ seconds, (2) divide totals $\rightarrow$ avg QPS, (3) $\times 2$ for peak, $\times$ ratio for reads, (4) count $\times$ bytes $\rightarrow$ storage, (5) QPS $\times$ payload $\rightarrow$ bandwidth. **Always state assumptions out loud.**

17.9 Observability & operations

Three pillars: **Logs** (discrete events — *why* it broke; carry a trace ID), **Metrics** (numeric time-series — rate, error %, p50/p95/p99 latency — cheap, drive **alerts**), **Traces** (a request's path across services — *where* latency is spent). *Metrics say something's wrong, traces say where, logs say why.* Tools: Prometheus/Grafana, OpenTelemetry/Jaeger.

Health checks: **liveness** ("process alive?" $\rightarrow$ restart) vs **readiness** ("can serve now?" $\rightarrow$ pull from LB without killing). Kubernetes/LBs poll these.

SLI / SLO / SLA: an **SLI** is a measured signal (e.g. % requests < 200 ms); an **SLO** is the internal target (e.g. 99.9% success/30 days) driving an **error budget** ($100\% - \text{SLO}$); an **SLA** is the external contractual promise with penalties. Keep SLO stricter than SLA.

Soundbite: "Metrics alert you that something's wrong, traces show where, logs show why — and you hold yourself to an SLO with an error budget that's stricter than the customer-facing SLA."

17.10 Resilience patterns (removing SPOFs)

A **single point of failure** is any un-redundant component whose failure kills the system. Remove via redundancy (replicas, multi-AZ/region), failover, and isolation. The patterns below stop *local* failures from cascading *globally*:

- **Timeouts** — bound every call; a slow dependency otherwise exhausts threads $\rightarrow$ cascade.
- **Retries with exponential backoff + jitter** — back off to avoid hammering a struggling service; **jitter** de-syncs clients to prevent a thundering herd. Cap retries.
- **Idempotent retries** — only retry safely if the op is idempotent, or you double-charge.
- **Circuit breaker** — after too many failures, "open" and fail fast for a cooldown, then "half-open" to test recovery. Gives the downstream room to heal.
- **Bulkhead** — isolate resource pools per dependency so one saturated dep doesn't drown the whole service.

- **Graceful degradation** — serve reduced functionality (stale cache, hide a widget) instead of erroring the whole request.
- **Load shedding / back pressure** — under overload, reject low-priority work early (429/503) to protect the core.

Soundbite: *"Bound every call with a timeout, retry transient failures with backoff + jitter idempotently, and wrap flaky dependencies in a circuit breaker + bulkhead so one slow service fails fast instead of cascading."*

Part V — Practice & Reference

Numbers every engineer should know

Powers of two (for storage/addressing math):

Power	Approx	Bytes
10	1 thousand	1 KB
20	1 million	1 MB
30	1 billion	1 GB
40	1 trillion	1 TB
50	1 quadrillion	1 PB

Latency numbers (Jeff Dean's list — memorize the *ratios*, not the digits):

Operation	Latency	Intuition
L1 cache reference	0.5 ns	—
Branch mispredict	5 ns	—
L2 cache reference	7 ns	14× L1
Mutex lock/unlock	25 ns	—
Main memory reference	100 ns	200× L1
Compress 1 KB (Zippy)	10 μs	—
Send 1 KB over 1 Gbps	10 μs	—
SSD random read	150 μs	~1 GB/s SSD
Read 1 MB seq from memory	250 μs	—
Round trip in same DC	500 μs	—
Read 1 MB seq from SSD	1 ms	4× memory
Disk seek	10 ms	20× DC round trip
Read 1 MB seq from disk	30 ms	120× memory
CA → Netherlands → CA	150 ms	speed of light tax

Derived rules of thumb: memory is ~100,000× faster than a disk seek · same-DC round trip ~0.5 ms ($\approx 2,000/\text{sec}$) · cross-continent RTT ~150 ms ($\approx 6\text{--}7/\text{sec}$) · read seq: disk ~30 MB/s, 1 Gbps net ~100 MB/s, SSD ~1 GB/s, memory ~4 GB/s.

Capacity numbers ("how much can one box take?"). These are *order-of-magnitude interview heuristics* — good to ± 1 order of magnitude only; real numbers swing 10× with hardware, row size, indexes, and query shape. Use them exactly like the

latency list: to justify "one box vs. many," never as precise SLAs.

Component	Comfortable single node	"Getting nervous" → scale out
SQL primary (writes)	~1,000 writes/s (10^3)	~5,000+/s → shard / queue / denormalize
SQL reads (with replicas)	~10,000 reads/s (10^4)	10^4 – 10^5 → add read replicas, cache
Redis / Memcached	~100,000 ops/s (10^5)	approaching 10^6 → shard the cache
NoSQL write node (Cassandra)	~10,000 writes/s per node	scales ~linearly — just add nodes
Kafka	~100k–1M msgs/s	rarely the bottleneck
App / web server (stateless)	~1,000–10,000 QPS	just add boxes behind an LB

The one to memorize: a single SQL primary tops out around 10^3 writes/s. This is the number the [Mint solution](#) leans on — 2,000 writes/s is "tough for a single master" → reach for federation / sharding / NoSQL. Notice every entry is a clean power of ten: memorize the **exponent, not the digits** (same discipline as the latency ratios). **How to use it:** chain estimate → threshold → design move — "I estimated ~2,000 writes/s; a single SQL master handles ~1,000, so I'm over budget → I need sharding or a write queue." If your estimate lands within ~2× of a threshold, say "borderline, I'd load-test it" rather than pretending the number is exact.

Availability nines: $99.9\% \approx 8.7 \text{ h/yr}$ · $99.99\% \approx 52 \text{ min/yr}$ · $99.999\% \approx 5 \text{ min/yr}$. In **series** multiply availabilities (worse); in **parallel** $1 - (1-a)(1-b)$ (better).

Source: [primer §Appendix](#).

Practice Problems

The repo ships 8 fully worked solutions. Below is a distilled version of each — the constraints, the core design, and **the one insight the interviewer is looking for**. For each, try solving it yourself with the [§2 framework](#) *before* reading the linked solution.

1. Pastebin / Bit.ly (URL shortener) → [solution](#)

- **Scale:** 10M users, ~40 reads/s, ~4 writes/s, 10:1 read:write, ~450 GB over 3 yr.
- **Design:** Write API generates a unique shortlink → stores metadata in **SQL** (shortlink → path), stores the paste **blob in an object store (S3)**, not the DB. Read API looks up SQL then fetches the blob. Analytics via **MapReduce over logs** (offline, not inline counts).
- **Insight:** URL generation = `base62(md5(ip + timestamp))[:7]`. Base62 ([A-Za-z0-9]) is URL-safe; `627` covers the keyspace. Separate metadata (SQL) from blob (object store), and keep analytics offline.
- **Scale-up:** cache popular reads + SQL read replicas; analytics → data warehouse.

2. Twitter timeline & search → [solution](#)

- **Scale:** 100M active, 500M tweets/day, 100K reads/s, 6K tweets/s, 60K fan-out deliveries/s.
- **Design:** posting a tweet stores it and calls a **Fan-Out Service** that pushes the tweet ID into each follower's home-timeline cache (Redis list, ~21 B/entry). Reads are then **O(1)** from cache, hydrated via Tweet/User info services. Search = scatter-gather over a Lucene cluster.
- **Insight: fan-out on write** (push) makes reads cheap by precomputing timelines. The trap the interviewer probes: the **celebrity problem** — millions of followers make fan-out slow and racy, so use a **hybrid** — don't fan out high-follower users; fetch their tweets at read time (pull) and merge.
- **Scale-up:** keep only active users' and recent tweets in cache; federate/shard SQL; NoSQL for volume.

3. Web crawler → [solution](#)

- **Scale:** 1B links, weekly refresh, ~1,600 writes/s, ~40K searches/s, 2 PB/month.
- **Design:** two NoSQL stores — `links_to_crawl` (priority queue, e.g. Redis sorted set) and `crawled_links` (URL → page signature). Loop: pop top link → check for a similar **signature** → if seen, demote priority & skip → else crawl, enqueue reverse-index + document jobs, add child URLs back.
- **Insight: content signatures** detect already-seen/near-duplicate pages → break crawl **cycles** and avoid re-crawling. Dedup URLs at scale with MapReduce; near-dup detection via Jaccard/cosine. Freshness via per-page timestamp + re-crawl interval + robots.txt.
- **Scale-up:** cache popular queries; shard reverse-index/document services; crawler keeps its own DNS cache (DNS is a bottleneck).

4. Mint.com (personal finance) → [solution](#)

- **Scale:** 10M users, 5B transactions/month, ~2,000 writes/s, **10:1 write:read** (write-heavy!).
- **Design:** connecting an account drops a job on a **queue**; a Transaction Extraction Service pulls it (bank fetches are slow/unreliable → async), stores raw logs in an object store, categorizes (seller → category dictionary + crowdsourced overrides), and aggregates monthly spend.
- **Insight:** two moves — (1) **async extraction** via queue decouples slow bank calls from the request; (2) **don't store what you can template/derive** — keep a generic budget template + store only user overrides, and compute `monthly_spending` via **MapReduce over the transaction log**.
- **Scale-up:** cache-aside + read replicas; aggregates → analytics DB; federate/shard + NoSQL for 2,000 writes/s.

5. Social network data structures → [solution](#)

- **Scale:** 100M users, 5B edges (graph too big for one machine), ~400 searches/s.
- **Design:** shortest path (degrees of separation) = **BFS** — but the graph is **sharded across Person Servers**, so each adjacency lookup is a network hop through a Lookup Service (person_id → server).
- **Insight: distributed BFS over a sharded graph.** BFS is trivial; the hard part is that edge traversals cross machines. Optimizations: **bidirectional BFS** (meet in the middle), **batch** friend lookups on the same server, **shard by location** (friends cluster geographically), cache traversals, cap hops.
- **Scale-up:** cache person data; precompute BFS offline into NoSQL.

6. Key-value cache for a search engine → [solution](#)

- **Scale:** 10M users, 10B queries/month, ~4,000 req/s, 270 B/entry.
- **Design:** Query API normalizes the query → checks an in-memory **LRU cache** (cache-aside) → on miss calls the reverse-index + document services and inserts at the front.
- **Insight: implement an LRU from scratch** = doubly-linked list (evict from tail) + hash map (O(1) access) → O(1) get/set/evict. This is half a coding question. Invalidation handled pragmatically with a **TTL**.
- **Scale-up:** shard the cache with `machine = consistent_hash(query)` (not per-machine caches, not full replicas).

7. Amazon sales rank by category → [solution](#)

- **Scale:** 10M products, 1,000 categories, 1B transactions/month, hourly updates, ~40K reads/s.
- **Design:** raw sales logs in an object store; a Sales Rank Service runs **MapReduce** and writes a small `sales_rank` aggregate table. Reads just hit that table.
- **Insight:** this is a **batch/analytics** problem, not a serving one → **MapReduce**, specifically a two-stage job whose second stage exploits the **shuffle/sort** to produce a distributed sort by putting `(category, quantity)` in the map key. Precompute offline; serve from a tiny table.
- **Scale-up:** cache popular ranks; raw data → analytics DB.

8. Scale to millions of users on AWS → [solution](#)

This one is the [S2 framework](#) applied iteratively — **the progression is the answer**. Each step is triggered by a *profiled bottleneck* (mantra: **benchmark** → **profile** → **fix the bottleneck** → **repeat**):

1. **Single box** (web + DB) + Elastic IP + DNS. Vertical-scale first.
 2. **Split the DB** onto its own host (RDS); move static content to **S3**. Now tiers scale independently.
 3. **Load balancer + multiple web servers** across AZs; **master-slave** MySQL failover; split web tier from app tier; add a **CDN**.
 4. **Fix read-heavy DB (100:1)**: add a **cache (ElastiCache)** and **read replicas**; move **sessions into the cache** → web tier becomes **stateless**.
 5. **Autoscaling** (enabled by statelessness) + full monitoring/DevOps automation.
 6. **Data-layer scaling + async**: warehouse for old data, **federation/sharding/denormalization**, **NoSQL** for specific data, **queues + workers** for non-realtime work (e.g. thumbnail generation).
- **Insight**: there's no single "answer architecture." The signal is that you **scale for a reason**, step by step, and that you know **statelessness (sessions in the cache) is what unlocks autoscaling**.

More questions to practice (with reference links) live in the [primer's additional-questions table](#): Dropbox/file sync, Google search, Google Docs (operational transform), Redis, recommendation systems, WhatsApp, Instagram, Facebook news feed, trending topics, a **Snowflake-style ID generator**, an **API rate limiter**, a stock exchange, and more.

Object-Oriented Design

Some interviews (especially at Amazon) include an OO/class-design round. The repo has 6 solved as Jupyter notebooks. Approach: clarify use cases → identify the core **classes/objects** and their relationships → define methods and interfaces → apply design patterns (strategy, factory, observer, state) where they fit → discuss extensibility.

Problem	Notebook	The core modeling challenge
Hash map	hash_map.ipynb	Buckets + collision handling (chaining).
LRU cache	lru_cache.ipynb	Doubly-linked list + hash map for O(1) — same trick as practice problem #6.
Call center	call_center.ipynb	Escalation hierarchy (operator → supervisor → director); dispatch.
Deck of cards	deck_of_cards.ipynb	Enums, inheritance, shuffle; extend to blackjack.
Parking lot	parking_lot.ipynb	Vehicle/spot type hierarchies + fitting rules — the classic OO question.
Online chat	online_chat.ipynb	Users, groups, messages; add/remove; send.

Anki decks for spaced repetition are in [resources/flash_cards/](#).

18. One-page cheat sheet

The framework: Scope (use cases, scale, read:write) → Estimate → API → Data model (SQL vs NoSQL + why) → High-level diagram → Deep-dive core → Identify bottlenecks → Scale each tier → Trade-offs.

Request path: Client → DNS → CDN → Load Balancer → Web/Reverse-proxy → App/Services → Cache ⇄ DB; slow work → Queue → Workers.

Scaling toolbox (in the order you usually reach for them): 1. **Vertical scale** (bigger box) — simplest, caps out, SPOF. 2.

Horizontal scale + load balancer — needs a **stateless** tier (sessions in Redis/DB). 3. **Cache** (cache-aside is the default) — biggest read win. 4. **CDN** — static/media near users. 5. **Read replicas** — scale reads. 6. **Federation** (split DBs by function) → **Sharding** (split rows, use consistent hashing) → **Denormalization** (kill joins). 7. **NoSQL** for the right data shape/scale. 8.

Async queues + workers — decouple slow/spiky work; watch back pressure.

Trade-off phrases to say out loud: - "This is read-heavy (100:1), so I'll optimize reads with caching + replicas." - "I'll trade strong consistency for availability here — a stale feed is fine, so AP/eventual consistency." - "Statelessness lets me autoscale the web tier." - "Fan-out on write makes reads cheap but breaks for celebrities — I'd go hybrid." - "I'll denormalize to avoid an expensive cross-shard join, accepting write amplification." - "That's a single point of failure; I'd add a standby / run it in parallel."

When X, reach for Y:

Symptom	Reach for
Read-heavy, hot keys	Cache (+ CDN), read replicas
Write-heavy ingest	LSM store (Cassandra), sharding, queue + batch
Expensive joins	Denormalization, materialized views
Slow inline work	Async queue + workers
Node churn reshuffles data	Consistent hashing
Duplicate messages	Idempotency keys / dedup
One slow dep stalls everything	Timeout + circuit breaker + bulkhead
Traffic spikes	Autoscaling (stateless tier) + back pressure
Need global agreement/config	Raft store (etcd/ZooKeeper)
Abuse / quota enforcement	Token bucket / sliding-window (Redis)

19. Self-test flashcard prompts

Cover the answers and recall out loud. (Deeper decks in [resources/flash_cards/](#).)

1. Performance problem vs scalability problem — what's the difference?
2. State CAP correctly. Why isn't P optional? What does PACELC add?
3. Weak vs eventual vs strong consistency — one use case each.
4. Two availability patterns? Series vs parallel availability formulas?
5. Push vs pull CDN — which for high traffic?
6. Layer 4 vs Layer 7 load balancing?
7. Why must horizontally-scaled web servers be stateless, and where do sessions go?
8. Five ways to scale an RDBMS.
9. Federation vs sharding?
10. The four NoSQL types + one example each. What's BASE?
11. SQL vs NoSQL — the decision checklist.
12. Cache-aside vs write-through vs write-behind — trade-offs.
13. Why is cache invalidation hard? Name a mitigation.
14. TCP vs UDP — when each?
15. REST vs RPC vs gRPC vs GraphQL — when each?
16. SSE vs WebSockets vs long polling?
17. Consistent hashing — the problem it solves; why only K/N keys move; what are vnodes?
18. B-tree vs LSM-tree — which is write-optimized and why?
19. Token bucket vs sliding-window counter?
20. Why is exactly-once "effectively-once"? How do idempotency keys work?
21. Quorum: what does $W + R > N$ guarantee?
22. Bloom filter: what's guaranteed, what isn't? One use.
23. Estimate QPS for 300M daily actions. ($\approx 3,500/s$)
24. Circuit breaker, bulkhead, backoff+jitter — what does each prevent?

25. SLI vs SLO vs SLA? What's an error budget?
26. Twitter fan-out: write vs read, and the celebrity fix.
27. How do you generate a short URL key, and how many base62 chars for ~4B keys? (6)

20. Study plan

Based on the [primer's timeline table](#). **You don't need to know everything** — go broad, then deep in a few areas.

Short timeline (breadth): - Read Parts I–III of this guide (framework + fundamentals + building blocks). - Watch the [Harvard scalability lecture](#) and skim the [AWS 10M-users notes](#). - Solve 2–3 practice problems (do Pastebin + Twitter + Scaling-on-AWS). - Memorize the [cheat sheet](#) and [latency numbers](#).

Medium timeline (breadth + some depth): - All of the above, plus Part IV deeper topics (consistent hashing, indexing, quorum, idempotency). - Solve most practice problems; do the estimation examples by hand. - Read 2–3 [real-world architectures](#) and the eng blogs of companies you're interviewing with. - Run the [Anki decks](#) daily.

Long timeline (breadth + more depth): - Everything above; solve *all* practice + [additional questions](#). - Read the seminal papers (below) — Dynamo, Bigtable, GFS, MapReduce, Spanner, Kafka. - Do mock interviews out loud; practice leading the conversation and drawing diagrams.

The day before: re-read the [cheat sheet](#), the [framework](#), and the trade-off phrases. Get sleep.

Video lecture notes

The primer suggests several video lectures. Distilled notes below so you get the lessons without watching all of them.

Harvard CS75 — Scalability (David Malan) · [watch](#)

Builds up scaling in the order you hit it as a site grows: - **Vertical scaling** (bigger box, RAID for disk redundancy/speed) — always hits a ceiling and is a SPOF. - **Horizontal scaling** — many commodity boxes → needs **load balancing** (round robin, load-aware, content-based; DNS round-robin as a cheap alternative). - **The session-state problem** (central theme): round-robin sends a logged-in user to a server that doesn't know them. Fixes: sticky sessions (uneven load, lose session on failure), address-in-cookie (leaks IPs), or — best — a **shared/centralized session store** so any server serves any request. - **Caching**: PHP opcode cache, MySQL query cache, **memcached** (in-RAM KV), static HTML. - **Replication**: master-slave (read scaling + redundancy) and master-master (write availability + conflict complexity). - **Partitioning/sharding** when one DB can't hold/serve it all. - **HA**: redundancy everywhere; pair every component (active-passive LBs, replicated DBs) — every component can fail.

AWS — Scaling to your first 10 million users · [watch](#)

The canonical "evolve as users grow" talk (matches practice problem #8): - **Start SQL** (RDS/Aurora) — proven, well-tooled; adopt NoSQL only for a specific need (huge write volume, flexible schema, no joins). - **Vertical-scale first** (cheap, simple), then **Multi-AZ** for redundancy. - **Add a load balancer + multiple web servers**; **make the web tier stateless** by pushing sessions to DynamoDB/ElastiCache — the key enabler for **autoscaling**. - **Scale reads** with read replicas + ElastiCache; offload static to **S3 + CloudFront**. - **Decouple** into services (SOA) + **queues (SQS)**; finally **federation + sharding + NoSQL** at millions of users. - Rules: scale vertically first, make everything stateless, use managed services, decouple, add complexity only when forced.

CAP theorem · [watch](#)

- The "pick 2 of 3" framing is misleading — **partitions are inevitable, so P is not optional**. The real choice (**CP vs AP**) only bites *during* a partition. When healthy, you get both C and A.
- Misconceptions: you don't *permanently* give one up; CAP says nothing about latency (that's what **PACELC** fixes — *else, latency vs consistency*).

- Classifications: Dynamo/Cassandra = **PA/EL**; RDBMS/Spanner = **PC/EC**.

Intro to systems design interviews (Gaurav Sen) · [watch](#)

A mental framework, not a specific system: **no single right answer**; **start simple, then scale**; **everything is a trade-off**. The loop: clarify functional + non-functional requirements → naive design → find the bottleneck → apply the next tool (vertical → LB + stateless horizontal → decouple/microservices → queues → replication/sharding/federation → caching/CDN) → address failure with redundancy. Statelessness unlocks horizontal scaling; discuss trade-offs out loud — that's the graded signal.

Intro to NoSQL (Martin Fowler) · [watch](#)

- Two drivers: **impedance mismatch** (object ↔ table friction) and **running on clusters** (the primary force — relational DBs assume one big machine).
- Four models: **key-value**, **document**, **column-family** (all *aggregate-oriented* → easy to distribute) and **graph** (the exception — optimized for traversing relationships).
- **Schemaless** ≠ structure-free — there's an *implicit* schema in the app code.
- NoSQL relaxes ACID → eventual consistency, **quorum** tuning. **NoSQL won't replace relational** — most apps should default to relational and choose NoSQL for a concrete reason.
- Headline: **polyglot persistence** — use the right store per job (relational for orders, KV for sessions, graph for the social network, document for the catalog).

Further reading map

Seminal papers (for depth): - [MapReduce](#) · [GFS](#) · [Bigtable](#) · [Dynamo](#) · [Spanner](#) · [Kafka](#) · [Dean](#) — building large distributed systems

Real-world architectures & blogs: the primer's [Real world architectures](#), [Company architectures](#), and [Company engineering blogs](#) tables — read the ones for companies you're interviewing with. For interviews, *identify shared principles and lessons learned*, don't memorize details.

Book (modern canonical text): *Designing Data-Intensive Applications* by Martin Kleppmann — the deepest single resource for everything in Part IV.

This repo: the full [README.md](#) (source of Parts I–III), the [solutions/](#) folder (worked problems), and [resources/flash_cards/](#) (Anki decks).

Built from [The System Design Primer](#) + its suggested video lectures + supplementary research. Remember the one idea: *everything is a trade-off — and your job in the interview is to say the trade-off out loud.*